Asincronía, promesas, `fetch` y modularización básica en JavaScript

> **Enfoque de la sesión**
>
> En esta sesión vas a trabajar uno de los cambios de mentalidad más importantes del bloque: **entender que no todos los datos están disponibles en el mismo instante en que los necesitas**. El objetivo no es solo aprender la sintaxis de promesas o de `fetch()`, sino comprender cómo se organiza el flujo de una aplicación cuando una operación tarda, cómo se espera correctamente su resultado y cómo conviene estructurar el código para que no quede todo mezclado.

1. Introducción

En la sesión anterior trabajaste con el DOM y con el renderizado dinámico. Eso te permitió construir interfaz a partir de datos, crear nodos, insertar contenido y reutilizar funciones de pintado. Sin embargo, hasta ahora la mayoría de los datos con los que has trabajado estaban disponibles desde el principio dentro del propio archivo JavaScript.

En una aplicación real eso no siempre ocurre.

Muy a menudo necesitas trabajar con información que:

- tarda en llegar;
- depende de una petición remota;
- se obtiene de una API;
- o simplemente no está disponible de forma inmediata.

Ese cambio es muy importante porque obliga a pensar de otra manera. Ya no basta con leer el código de arriba abajo suponiendo que cada línea deja el dato preparado para la siguiente. En muchos casos, el dato **llega después**.

Por eso esta sesión es especialmente importante. Aquí vas a trabajar:

- la diferencia entre sincronía y asincronía;
- el papel de `setTimeout()` como primer ejemplo visible;
- el modelo mental del event loop;
- callbacks asíncronos;
- promesas;

- `.then()`, `.catch()` y `.finally()`;
- `Promise.all()`;
- `async` y `await`;
- `fetch()` como primer contacto real con datos externos;
- y una primera organización del código separando acceso a datos, transformación y renderizado.

Esta sesión actúa como un punto de inflexión dentro del bloque. Hasta ahora sabías **pintar datos**. A partir de aquí empiezas a trabajar con el hecho de que los datos pueden **llegar más tarde** y de que el código debe estar preparado para ello.

---

2. Sincronía y asincronía

2.1. Qué significa ejecución síncrona

En un flujo síncrono, las instrucciones se ejecutan en orden y cada una termina antes de que empiece la siguiente.

```
console.log("Inicio");
console.log("Proceso");
console.log("Fin");
```

La salida seguirá exactamente ese orden.

2.2. Qué significa ejecución asíncrona

En muchas aplicaciones reales hay operaciones cuyo resultado no está disponible en el momento en que se solicitan. Eso ocurre, por ejemplo, cuando:

- esperas una respuesta de red;
- programas una acción para más tarde;
- trabajas con un temporizador;
- cargas un recurso externo.

La asincronía describe precisamente ese escenario: **la operación se inicia ahora, pero el resultado llega después**.

2.3. Por qué esto cambia la forma de pensar

Cuando trabajas de forma síncrona, sueles asumir que una instrucción deja preparado el dato para la siguiente. En asincronía eso no siempre es cierto.

Idea clave

La asincronía no significa simplemente “hacer muchas cosas a la vez”.
Significa gestionar correctamente operaciones cuyo resultado no está disponible en el mismo instante en que se solicitan.

Referencias específicas del subtema

- MDN Web Docs — Asynchronous JavaScript
- MDN Web Docs — Introduction to asynchronous JavaScript

---

3. Primer contacto con `setTimeout()`

3.1. Qué hace

`setTimeout()` permite ejecutar una función después de un tiempo determinado.

```
console.log("Inicio");

setTimeout(() => {
  console.log("Mensaje retrasado");
}, 2000);

console.log("Fin");
```

3.2. Qué observas aquí

Aunque `setTimeout()` esté escrito antes del último `console.log()`, el mensaje retrasado aparece después.

El orden será:

1. `Inicio`
2. `Fin`
3. `Mensaje retrasado`

3.3. Qué enseña este ejemplo

Este ejemplo ya muestra una idea decisiva:

- el código no se comporta siempre como una simple lectura lineal del archivo;

- algunas operaciones quedan pendientes;
- y su ejecución efectiva ocurre más tarde.

3.4. Qué no debes interpretar mal

`setTimeout()` no “detiene” toda la aplicación hasta que pasa el tiempo. Lo que hace es programar una función para más adelante.

Referencias específicas del subtema

- MDN Web Docs — `setTimeout()`

---

4. El event loop: por qué JavaScript puede esperar sin bloquear toda la aplicación

4.1. El problema que intenta resolver

Cuando empiezas a trabajar con temporizadores, promesas o peticiones remotas, aparece una pregunta natural: si JavaScript ejecuta el código en un hilo principal, ¿cómo puede seguir funcionando la aplicación mientras algunas operaciones esperan?

La respuesta general está en cómo se coordinan:

- el **call stack**;
- ciertas colas de tareas;
- y el **event loop**.

4.2. El call stack

El **call stack** es la pila donde JavaScript ejecuta el código síncrono y las funciones que entran en ejecución en ese momento.

Mientras el stack está ocupado, otras tareas no lo interrumpen a mitad del proceso.

4.3. El papel del entorno del navegador

Operaciones como:

- `setTimeout()`,
- eventos del navegador,
- `fetch()`,

no se resuelven completamente dentro del stack principal. El entorno del navegador gestiona parte de ese trabajo y, cuando la operación está lista para continuar, su continuación espera el momento adecuado para volver al flujo principal.

4.4. Task queue y microtask queue

De forma simplificada, conviene distinguir entre dos colas:

- **task queue**: donde suelen esperar tareas como callbacks de `setTimeout()`;
- **microtask queue**: donde se colocan, por ejemplo, las resoluciones de promesas.

4.5. Qué hace el event loop

El **event loop** revisa continuamente si el call stack está libre. Cuando lo está, puede tomar tareas pendientes de las colas y llevarlas a ejecución.

Dicho de forma simple:

1. se ejecuta el código síncrono;
2. ciertas operaciones quedan delegadas;
3. cuando terminan, esperan en una cola;
4. el event loop comprueba si el stack está libre;
5. entonces introduce la siguiente tarea que corresponda.

4.6. Ejemplo importante

```
console.log("Inicio");

setTimeout(() => {
  console.log("Timeout");
}, 0);

Promise.resolve().then(() => {
  console.log("Promesa");
});

console.log("Fin");
```

La salida será:

1. `Inicio`
2. `Fin`
3. `Promesa`

4. `Timeout`

4.7. Cómo interpretar ese orden

- `Inicio` y `Fin` son síncronos, así que se ejecutan primero.
- La resolución de la promesa entra en la **microtask queue**.
- El callback de `setTimeout()` entra en la **task queue**.
- Cuando el stack queda libre, las microtareas se atienden antes que las tareas normales.

> **Idea clave**
>
> El orden del código escrito no siempre coincide con el orden final de ejecución cuando interviene la asincronía.

4.8. Qué debes retener

No necesitas memorizar una teoría exhaustiva del motor de JavaScript. Lo importante aquí es entender estas ideas:

- el código síncrono se ejecuta en el call stack;
- algunas operaciones se delegan al entorno;
- cuando terminan, esperan en colas;
- el event loop coordina cuándo vuelven al flujo principal;
- y no todas las tareas pendientes se atienden con la misma prioridad.

Referencias específicas del subtema

- MDN Web Docs — JavaScript execution model
- MDN Web Docs — `setTimeout()`
- MDN Web Docs — `Promise`

---

5. Callbacks asíncronos

5.1. Qué relación tienen con la asincronía

Ya conoces los callbacks como funciones que se pasan a otras funciones. En asincronía, aparecen cuando una función debe ejecutarse más tarde, al completarse una operación.

```
setTimeout(() => {
  console.log("Esta función se ejecuta después");
}, 1000);
```

5.2. Qué problema pueden generar

Cuando varias operaciones asíncronas se anidan en exceso, el código puede volverse difícil de leer y mantener.

```
setTimeout(() => {
  console.log("Paso 1");

  setTimeout(() => {
    console.log("Paso 2");

    setTimeout(() => {
      console.log("Paso 3");
    }, 1000);
  }, 1000);
}, 1000);
```

5.3. Por qué importa entender esto

No se trata de demonizar los callbacks, sino de comprender por qué JavaScript ofrece herramientas más expresivas para muchos flujos asíncronos.

Referencias específicas del subtema

- MDN Web Docs — Callback function
- MDN Web Docs — `setTimeout()`

---

6. Promesas: representar resultados futuros

6.1. Qué problema intentan resolver

Cuando una operación tarda, necesitas una forma de representar que el resultado llegará más tarde. Las promesas son una de las herramientas centrales de JavaScript para ello.

6.2. Idea general

Una promesa representa una operación asíncrona que puede estar en uno de estos estados:

- pendiente;
- resuelta;
- rechazada.

6.3. Ejemplo básico

```
const operacion = new Promise((resolve) => {
  setTimeout(() => {
    resolve("Resultado disponible");
  }, 1000);
});

operacion.then((resultado) => {
  console.log(resultado);
});
```

6.4. Qué ocurre aquí

- se crea una promesa;
- tras un segundo, se resuelve con un valor;
- `.then()` permite reaccionar cuando el valor ya está disponible.

6.5. Qué aporta frente al callback anidado

Las promesas permiten expresar el flujo con más claridad y encadenar mejor operaciones que dependen unas de otras.

> **Idea clave**
>
> Una promesa no es el dato final.
> Es la representación de una operación cuyo resultado llegará más tarde.

Referencias específicas del subtema

- MDN Web Docs — `Promise`
- MDN Web Docs — Using promises

---

7. `.then()`, `.catch()` y `.finally()`

7.1. `.then()`

`.then()` se usa para trabajar con el resultado cuando la promesa se resuelve correctamente.

```
const promesa = Promise.resolve("Todo correcto");

promesa.then((resultado) => {
  console.log(resultado);
});
```

7.2. `.catch()`

`.catch()` permite capturar errores.

```
const promesa = Promise.reject(new Error("Algo ha fallado"));

promesa.catch((error) => {
  console.error(error.message);
});
```

7.3. `.finally()`

`.finally()` permite ejecutar lógica que debe ocurrir siempre, independientemente del resultado final.

```
const promesa = Promise.resolve("Dato cargado");

promesa
  .then((resultado) => {
    console.log(resultado);
  })
  .catch((error) => {
    console.error(error);
  })
  .finally(() => {
    console.log("La operación ha terminado");
  });
```

7.4. Qué aporta esta estructura

Permite distinguir con claridad:

- qué ocurre si todo va bien;
- qué ocurre si algo falla;
- qué debe ejecutarse siempre al final.

Referencias específicas del subtema

- MDN Web Docs — `Promise.then()`
- MDN Web Docs — `Promise.catch()`
- MDN Web Docs — `Promise.finally()`

---

8. `Promise.all()` como primera aproximación a concurrencia

8.1. Qué resuelve

`Promise.all()` permite esperar a que varias promesas se resuelvan antes de continuar.

8.2. Ejemplo simple

```
const promesa1 = Promise.resolve("Dato 1");
const promesa2 = Promise.resolve("Dato 2");

Promise.all([promesa1, promesa2]).then((resultados) => {
  console.log(resultados);
});
```

8.3. Cuándo resulta útil

Es útil cuando:

- necesitas varios resultados a la vez;
- la lógica posterior depende de que todas las operaciones terminen;
- quieres coordinar varias tareas asíncronas.

8.4. Qué debes retener aquí

En esta fase basta con comprender su papel general. Más adelante podrás usarlo con más intención cuando las operaciones remotas sean más ricas.

Referencias específicas del subtema

- MDN Web Docs — `Promise.all()`

---

9. `async` y `await`

9.1. Qué aportan

`async` y `await` permiten escribir código asíncrono con una sintaxis más legible en muchos casos.

9.2. Ejemplo inicial

```
function esperarUnSegundo() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("Ha pasado un segundo");
    }, 1000);
  });
}

async function mostrarMensaje() {
  const mensaje = await esperarUnSegundo();
  console.log(mensaje);
}

mostrarMensaje();
```

9.3. Qué debes entender bien

- `await` solo puede usarse dentro de una función `async`;
- `await` no convierte mágicamente una operación en síncrona;
- simplemente permite expresar la espera de forma más clara dentro de la función.

9.4. Relación con el event loop y las promesas

Aunque `async` y `await` hacen el código más legible, por debajo sigues trabajando con promesas y con el mismo modelo de asincronía que ya has visto.

> **Idea clave**
>
> `async` y `await` mejoran la legibilidad del flujo, pero no eliminan la naturaleza asíncrona de la operación.

Referencias específicas del subtema

- MDN Web Docs — `async function`
- MDN Web Docs — `await`

10. Qué es una API y por qué importa en frontend

10.1. Idea general

Una API (*Application Programming Interface*) es un mecanismo que permite que un programa se comunique con otro. En frontend, lo habitual es consumir APIs que devuelven información para ser mostrada o utilizada en una interfaz.

10.2. Qué tipo de datos pueden recibirse

Una API puede devolver, por ejemplo:

- usuarios;
- productos;
- publicaciones;
- recursos individuales;
- colecciones completas;
- datos meteorológicos;
- resultados de búsqueda.

10.3. Qué cambia respecto a trabajar con datos locales

Hasta ahora muchas colecciones podían estar escritas directamente en tu archivo:

```
const productos = [
  { nombre: "Teclado", precio: 30 },
  { nombre: "Ratón", precio: 15 }
];
```

Cuando trabajas con una API, el flujo cambia:

1. la aplicación solicita información a una URL;
2. la API responde;
3. tu código interpreta esa respuesta;
4. la interfaz se renderiza a partir de esos datos.

Idea clave

En una aplicación conectada a una API, los datos no siempre existen desde el principio dentro del código.
Muchas veces **llegan después**.

Referencias específicas del subtema

- MDN Web Docs — Fetch API
- MDN Web Docs — Working with JSON
- MDN Web Docs — HTTP overview

---

11. `fetch()` : solicitar recursos remotos

11.1. Qué es `fetch()`

`fetch()` es una función nativa del navegador que permite realizar peticiones HTTP y trabajar con la respuesta de forma asíncrona.

11.2. Ejemplo básico

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then((response) => response.json())
  .then((usuarios) => {
    console.log(usuarios);
  })
  .catch((error) => {
    console.error("Error al cargar usuarios:", error);
  });
```

11.3. Qué ocurre aquí

- `fetch()` inicia la petición;
- devuelve una promesa;
- cuando la respuesta llega, se procesa;
- `response.json()` interpreta el cuerpo;
- y el resultado final se obtiene después.

11.4. Qué devuelve realmente `fetch()`

No devuelve directamente los datos finales, sino una **promesa**.

> ⚠ **Error conceptual frecuente**
>
> `fetch()` no te entrega “los datos ya listos” de forma inmediata.
> Te entrega una operación asíncrona cuyo resultado debe esperar.

Referencias específicas del subtema

- MDN Web Docs — Fetch API
- MDN Web Docs — `fetch()`

---

12. Primer uso de `fetch()` con `async` y `await`

12.1. Ejemplo básico

```
async function cargarUsuarios() {
  const response = await fetch("https://jsonplaceholder.typicode.com/users");
  const usuarios = await response.json();

  console.log(usuarios);
}

cargarUsuarios();
```

12.2. Qué está ocurriendo

- la función se declara con `async`;
- `await` espera la respuesta;
- `await` vuelve a usarse para interpretar el JSON;
- solo después tienes disponible la estructura JavaScript final.

12.3. Qué debes retener en esta fase

En esta sesión no necesitas profundizar todavía en todo el detalle de respuestas HTTP o errores avanzados. Lo importante aquí es consolidar:

- que `fetch()` es una operación asíncrona;
- que primero obtienes una respuesta;
- que después transformas su cuerpo;
- y que solo entonces puedes trabajar con los datos.

Referencias específicas del subtema

- MDN Web Docs — `fetch()`
- MDN Web Docs — `Response.json()`
- MDN Web Docs — `async function`
- MDN Web Docs — `await`

---

13. Transformar datos remotos antes de renderizarlos

13.1. Por qué conviene hacerlo

Cuando recibes información remota, no siempre coincide exactamente con la estructura que necesita tu interfaz. Por eso resulta útil adaptar los datos antes de pintarlos.

13.2. Ejemplo simple

```
async function cargarUsuarios() {
  const response = await fetch("https://jsonplaceholder.typicode.com/users");
  const usuarios = await response.json();

  const usuariosAdaptados = usuarios.map((usuario) => {
    return {
      nombre: usuario.name,
      correo: usuario.email
    };
  });

  console.log(usuariosAdaptados);
}

cargarUsuarios();
```

13.3. Qué enseña este ejemplo

- la API puede devolver más información de la que necesitas;
- muchas veces conviene construir una versión más útil para tu interfaz;
- separar esta adaptación mejora el diseño del código.

Referencias específicas del subtema

- MDN Web Docs — `Array.prototype.map()`

- MDN Web Docs — Fetch API

---

14. Renderizar datos remotos

14.1. Flujo general

Una vez que recibes y adaptas los datos, el siguiente paso habitual es renderizarlos en el DOM.

14.2. Ejemplo de renderizado

```
<section id="lista-usuarios"></section>
```

```
const listaUsuarios = document.querySelector("#lista-usuarios");

function renderizarUsuarios(usuarios) {
  listaUsuarios.innerHTML = "";

  usuarios.forEach((usuario) => {
    const tarjeta = document.createElement("article");
    const nombre = document.createElement("h2");
    const correo = document.createElement("p");

    nombre.textContent = usuario.nombre;
    correo.textContent = usuario.correo;

    tarjeta.append(nombre, correo);
    listaUsuarios.append(tarjeta);
  });
}
```

14.3. Qué se integra aquí

En este punto confluyen ideas de varias sesiones:

- arrays;
- métodos de arrays;
- DOM;
- renderizado dinámico;
- asincronía;
- `fetch()`.

Referencias específicas del subtema

- MDN Web Docs — `Document.createElement()`
- MDN Web Docs — `Element.append()`

---

15. Separar acceso a datos, transformación y renderizado

15.1. Una organización mínima recomendable

A medida que una miniaplicación crece, no conviene dejar todo en una única función enorme. Una primera estructura razonable consiste en distinguir tres piezas:

- función que obtiene datos;
- función que transforma datos;
- función que renderiza.

15.2. Ejemplo conceptual

```
async function obtenerUsuarios() {
  const response = await fetch("https://jsonplaceholder.typicode.com/users");
  return await response.json();
}

function transformarUsuarios(usuarios) {
  return usuarios.map((usuario) => ({
    nombre: usuario.name,
    correo: usuario.email
  }));
}

function renderizarUsuarios(usuarios) {
  console.log("Renderizando usuarios:", usuarios);
}
```

15.3. Qué aporta esta separación

- el acceso a datos queda aislado;
- la transformación puede comprobarse mejor;
- el renderizado no necesita conocer toda la estructura original de la API;
- el código se vuelve más mantenible.

Idea clave

Aunque la aplicación sea pequeña, separar responsabilidades desde el principio mejora mucho la claridad del código.

Referencias específicas del subtema

- MDN Web Docs — Functions
- MDN Web Docs — Fetch API

---

16. Modularización básica con `export` e `import`

16.1. Por qué modularizar

A medida que una miniaplicación crece, no conviene que todo el código esté en un único archivo. Separar responsabilidades mejora la claridad y la mantenibilidad.

16.2. Qué tipo de piezas puedes separar

Por ejemplo:

- acceso a la API;
- utilidades de transformación;
- renderizado;
- lógica principal.

16.3. Exportación nombrada

Archivo `api.js`:

```
export async function obtenerUsuarios() {
  const response = await fetch("https://jsonplaceholder.typicode.com/users");
  return await response.json();
}
```

Archivo `main.js`:

```
import { obtenerUsuarios } from "./api.js";

async function iniciar() {
```

```
  const usuarios = await obtenerUsuarios();
  console.log(usuarios);
}

iniciar();
```

16.4. Qué debes retener

En esta sesión no se trata de convertir módulos en un bloque enorme y aislado, sino de entender una idea muy práctica:

- un archivo puede **exportar** funciones o valores;
- otro archivo puede **importarlos**;
- así distribuyes mejor responsabilidades.

> **Idea clave**
>
> Modularizar no consiste en “partir por partir”, sino en separar partes del programa con responsabilidades claras.

Referencias específicas del subtema

- MDN Web Docs — JavaScript modules
- MDN Web Docs — `export`
- MDN Web Docs — `import`

---

17. Análisis técnico avanzado

17.1. La asincronía no es solo una sintaxis nueva

Uno de los mayores retos de esta sesión no es aprender a escribir `await`, sino aceptar que el programa ya no puede pensarse siempre como una simple secuencia lineal donde cada línea deja el dato preparado para la siguiente.

17.2. Entender el event loop evita que la asincronía parezca magia

Aunque en esta fase no se exige una teoría exhaustiva del motor de JavaScript, comprender que existen un stack, colas y un mecanismo de coordinación ayuda mucho a interpretar correctamente promesas, temporizadores y `fetch()`.

17.3. Promesas y `async/await` expresan un mismo fondo conceptual

La sintaxis cambia, pero la idea de base sigue siendo la misma: gestionar operaciones cuyo resultado no está disponible en el mismo momento en que se solicitan.

17.4. Traer datos desde fuera obliga a organizar mejor el código

Cuando una colección deja de ser local y pasa a venir de una API, se vuelve más evidente la necesidad de distinguir entre obtención, transformación y renderizado. Esa organización es importante no solo aquí, sino también como preparación para React.

17.5. Modularizar antes de React tiene mucho valor

El alumnado va a encontrarse módulos desde el primer momento en React. Por eso conviene llegar a ese bloque habiendo visto ya que:

- el código puede distribuirse en varios archivos;
- un archivo puede exportar funciones;
- otro puede importarlas;
- y cada parte puede tener una responsabilidad diferenciada.

Referencias específicas del subtema

- MDN Web Docs — JavaScript execution model
- MDN Web Docs — `Promise`
- MDN Web Docs — Fetch API
- MDN Web Docs — JavaScript modules

---

18. Ejemplo integrador de sesión

A continuación tienes un ejemplo que reúne varios conceptos trabajados.

```
<section id="lista-usuarios"></section>
```

```
const listaUsuarios = document.querySelector("#lista-usuarios");

function transformarUsuarios(usuarios) {
  return usuarios.map((usuario) => {
    return {
      nombre: usuario.name,
```

```
      correo: usuario.email
    };
  });
}

function renderizarUsuarios(usuarios) {
  listaUsuarios.innerHTML = "";

  usuarios.forEach((usuario) => {
    const tarjeta = document.createElement("article");
    const nombre = document.createElement("h2");
    const correo = document.createElement("p");

    nombre.textContent = usuario.nombre;
    correo.textContent = usuario.correo;

    tarjeta.append(nombre, correo);
    listaUsuarios.append(tarjeta);
  });
}

async function obtenerUsuarios() {
  const response = await fetch("https://jsonplaceholder.typicode.com/users");
  return await response.json();
}

async function iniciar() {
  const usuarios = await obtenerUsuarios();
  const usuariosAdaptados = transformarUsuarios(usuarios);
  renderizarUsuarios(usuariosAdaptados);
}

iniciar();
```

Qué se está aplicando aquí

Elemento	Uso en el ejemplo
`fetch()`	obtención de datos externos
`await`	espera de respuesta y de JSON
Métodos de arrays	adaptación de la colección remota
Renderizado dinámico	creación de tarjetas en el DOM

Elemento	Uso en el ejemplo
Separación de responsabilidades	acceso a datos, transformación y renderizado diferenciados

---

19. Errores frecuentes al comenzar

> ⚠ **Errores muy habituales**
>
> Estos fallos aparecen con frecuencia al empezar a trabajar con asincronía, promesas y datos remotos.

19.1. Pensar que todo el código se ejecuta de manera estrictamente lineal e inmediata

Los temporizadores, promesas y operaciones de red rompen esa intuición inicial.

19.2. Tratar una promesa como si ya fuese el dato final

Una promesa representa una operación en curso, no el valor listo para usarse inmediatamente.

19.3. Olvidar `await` o no encadenar correctamente la promesa

Si no esperas la operación, intentarás usar los datos antes de que estén disponibles.

19.4. Tratar `fetch()` como si devolviera directamente el array u objeto final

`fetch()` devuelve primero una promesa de respuesta, no el resultado final ya interpretado.

19.5. Mezclar petición, transformación y renderizado en una función demasiado grande

Aunque en ejemplos pequeños pueda parecer cómodo, esa mezcla complica mucho la lectura y la reutilización.

19.6. Pensar que modularizar es solo “dividir archivos”

Lo importante no es repartir líneas al azar, sino separar responsabilidades claras.

---

20. Síntesis de la sesión

En esta sesión has trabajado uno de los cambios conceptuales más importantes del bloque:

- diferencia entre sincronía y asincronía;
- uso de `setTimeout()` como primer ejemplo visible;
- noción básica de event loop;
- callbacks asíncronos;
- promesas;
- `.then()`, `.catch()` y `.finally()`;
- `Promise.all()`;
- `async` y `await`;
- primer contacto real con `fetch()`;
- idea de API y de datos remotos;
- transformación de datos externos;
- separación entre acceso a datos, transformación y renderizado;
- modularización básica con `export` e `import`.

Esta sesión es especialmente importante porque te obliga a cambiar la forma de pensar el flujo del programa y porque prepara directamente el terreno para trabajar con datos externos, con organización modular del código y con React.

✓ **Al terminar esta sesión deberías ser capaz de...**

- explicar qué significa que una operación sea asíncrona;
- interpretar el papel general de `setTimeout()` y del event loop;
- reconocer qué representa una promesa;
- usar `.then()`, `.catch()` y `.finally()`;
- comprender la utilidad de `Promise.all()`;
- utilizar `async` y `await` con sentido básico;
- explicar qué hace `fetch()` y por qué no devuelve el dato final de inmediato;
- adaptar datos remotos antes de renderizarlos;
- separar el acceso a datos del renderizado;
- comprender qué aporta una modularización mínima con `export` e `import`.

---

21. Referencias documentales generales

Referencias oficiales y de consulta

- MDN Web Docs — Asynchronous JavaScript
- MDN Web Docs — Introduction to asynchronous JavaScript
- MDN Web Docs — JavaScript execution model
- MDN Web Docs — `setTimeout()`
- MDN Web Docs — `Promise`
- MDN Web Docs — `Promise.then()`
- MDN Web Docs — `Promise.catch()`
- MDN Web Docs — `Promise.finally()`
- MDN Web Docs — `Promise.all()`
- MDN Web Docs — `async function`
- MDN Web Docs — `await`
- MDN Web Docs — Fetch API
- MDN Web Docs — `fetch()`
- MDN Web Docs — `Response.json()`
- MDN Web Docs — Working with JSON
- MDN Web Docs — JavaScript modules
- MDN Web Docs — `export`
- MDN Web Docs — `import`
- MDN Web Docs — Functions

Recomendación de estudio

Al repasar esta sesión, no te limites a memorizar sintaxis. Intenta reconstruir mentalmente el flujo: qué parte se ejecuta inmediatamente, qué parte queda pendiente, cuándo llega realmente el dato y cómo se conecta después con la transformación y con el renderizado. Comprender bien ese recorrido es una de las mejores preparaciones posibles para trabajar con APIs reales, con modularización y con React.